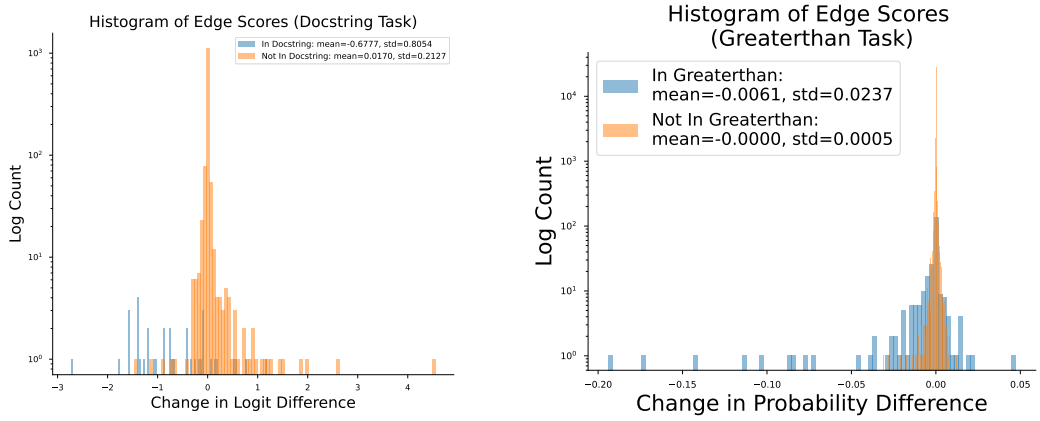
B Distribution of EAP Attribution Scores



(a) Distribution of Attribution Scores for the Docstring Task

(b) Distribution of Attribution Scores for the Greater-Than Task

Figure 7: Distribution of Attribution Scores for the Docstring and Greater-Than tasks

C Further investigation into combining EAP with ACDC

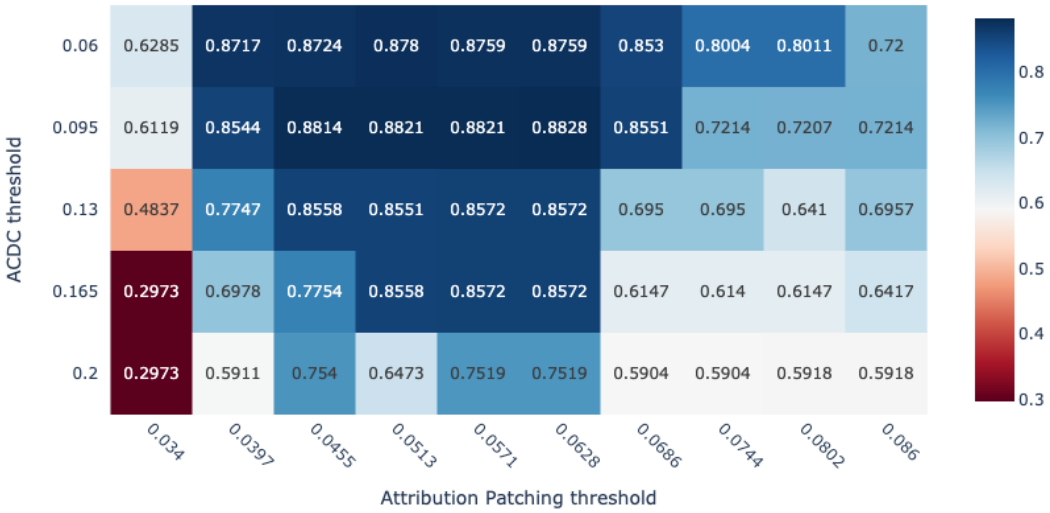


Figure 8: Youdens-J statistic (maximum TPR minus FPR value) for combining EAP and ACDC methods on the docstring task. We applied ACDC to the pruned subgraph returned by EAP.

D Further failures of attribution patching approximation

In Figure 9 we show further cases where in the docstring task attribution patching can be misleading. These cases all involve an edge that comes from the model's embeddings (positional and tokens). Our interpretation is that weighted averages of embeddings are anomalous inputs to the model and cause the concave change in docstring logit diff which doesn't occur when edges ae between non-embedding model components.

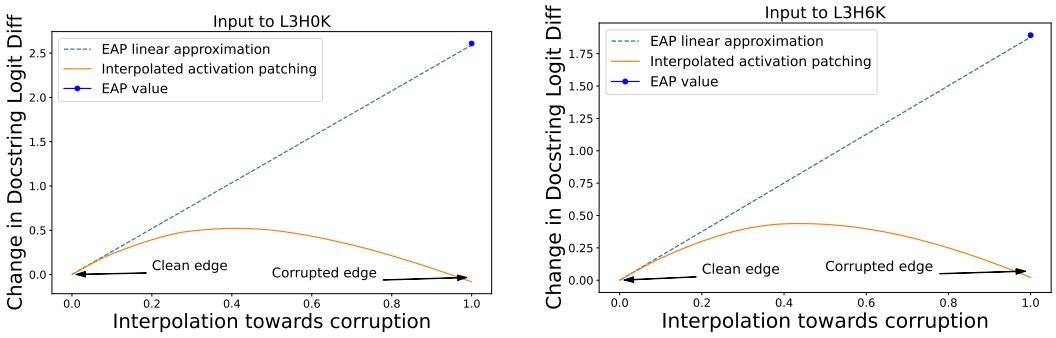


Figure 9: Visualizing Edge Attribution Patching in two further cases where the concave activation patching curve means the linear fit is poor.

E Edges Roles in IOI

We further explore the attribution scores for the IOI circuit. The IOI circuit is comprised of different attention head classes such as Induction heads, S-Inhibition heads, etc. [3]. Figure 10 shows the distributions of scores stratified by the roles of the edges. The edge roles are defined according to the role of their origin node. While edge roles such as Previous Token, Duplicate Token, Induction, and S-Inhibition edges have attribution scores centered around zero, we see a bias in edge scores given to name mover and negative name mover edges. As the name mover edges are directly responsible for the model outputting the indirect object, the attribution scores are largely negative since ablating these edges removes the model's ability to output the indirect object, lowering the logit difference. Similarly, the negative name movers have attribution scores that are largely positive since ablating these edges improves the logit difference. This matches the intuitive function of the edges.

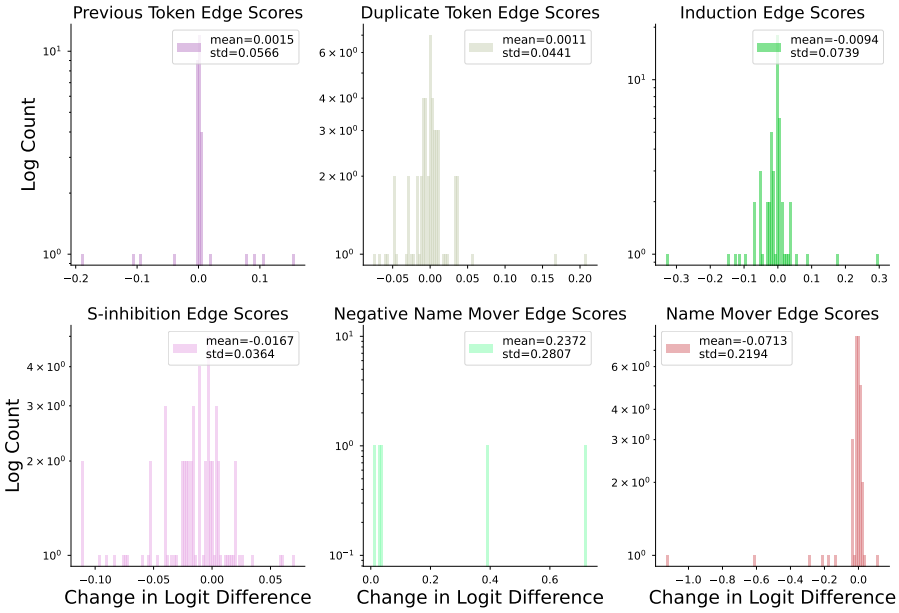


Figure 10: Distribution of Attribution Scores for each Edge Role in the IOI Task.

F Only one backwards pass is required for EAP

Note: it may be easier to understand our implementation https://github.com/Aaquib111/acdcpp/blob/main/utils/prune_utils.py#L249 rather than read this explanation. Alternatively, this derivation uses essentially the same arguments as Nanda [20][4] though with an updated codebase.

There are only two types of edges iterated over in ACDC: i) residual edges where the result is added at its endpoint, and ii) edges between the residual stream and the query, key and value calculations.[5] Clearly for all edges like ii) we can compute the gradient terms in Equation (2) in one backwards pass.

Interestingly, for all $\Delta_e L$ terms where e is a type i) edge (i.e added at the endpoint), we only need calculate the gradient with respect to the endpoint of the edge! For example, suppose we're calculating the effect of L0H0 on L1H0Q. If we represent the input to L1H0Q as a node V in the computational graph then

$$\frac{\partial}{\partial e_{\text{clean}}} L(x_{\text{clean}} | \text{ do}(E = e_{\text{clean}})) = \frac{\partial}{\partial v_{\text{clean}}} L(x_{\text{clean}} | \text{ do}(V = v_{\text{clean}})) \tag{3}$$

due to how V is just the sum of all the edges entering V. This allows efficient calculation of all the $\Delta_e L$ values since gradients with respect to nodes in computational graphs are calculated by default in backwards passes.

[4] Specifically, this section: https://www.neelnanda.io/mechanistic-interpretability/attribution-patching#how-to-think-about-activation-patching=:~:text=axes%20of%20variation.-,Path%20patching,-The%20core%20intuition

[5] It may be worth looking at some ACDC outputs from [10]. See https://colab.research.google.com/github/ArthurConmy/Automatic-Circuit-Discovery/blob/main/notebooks/colabs/ACDC_Implementation_Demo.ipynb for an explanation of this design choice.